IBM Data Analyst Capstone

Introduction

You have recently been hired as a Data Analyst by a global IT and business consulting firm renowned for its expertise in IT solutions and highly experienced consultants. To stay competitive in the rapidly evolving technology landscape, your organization regularly analyzes data to identify emerging and future skill requirements.

Roles and responsibilities:

As a Data Analyst, you will contribute to this initiative by collecting data from diverse sources and identifying trends for this year's report on in-demand skills. One key source for your analysis will be the latest Developer Survey, a comprehensive dataset offering insights into the global developer community.
Your initial task is to gather data on the most sought-after programming skills from various sources, including:

- Job postings
- Training portals
- Developer surveys, such as the latest Stack Overflow Developer Survey

After collecting sufficient data, you will analyze it to identify key insights and trends. Some of the trends you will explore include:

- Which programming languages are most in demand?
- Which database technologies are currently most sought after?
- Which Integrated Development Environments (IDEs) are the most popular?

You will begin by scraping internet websites, accessing APIs, and working with datasets like the latest Stack Overflow Developer Survey in various formats such as .csv files, Excel sheets, and databases.
After gathering the data, you will apply data-wrangling techniques to prepare the data for analysis.
Once the data is prepared, you will employ statistical techniques to analyze the data, identifying key trends and insights. You will then synthesize your findings using IBM Cognos Analytics to create a dashboard that visualizes the results. Finally, you will share your insights through a presentation, showcasing your storytelling and data analysis skills.

Evaluation:

Throughout the course, you will be evaluated using quizzes in each module as well as through the final project presentation.

Review
Of Accessing APIs

Estimated time needed: 30 minutes

O
bjectives

After completing this lab, you will be able to:
Understand HTTP
Analyze HTTP Requests

Table of Contents

Overview of HTTP

When you, the client, access a web page, your browser sends an HTTP request to the server where the page is hosted. The server tries to locate the desired resource by default "index.html **". If your request is successf ul, the server will send the object to the client in an HTTP response, which includes information like the type of the resource, the length of the resource, and other information.**
The figure below represents the process. The circle on the left represents the client, the circle on the right represents the web server. The table under the web server represents

_
Requests
HTTP
_

a list of resources stored in the web server. In this case an HTML **file,** png **image, and** txt **file .**
The HTTP protocol enables you to send and receive information through the web including webpages, images, and other web resources. In this lab, you will explore the Requests library for interacting with the HTTP **protocol.**

Uniform Resource Locator (U R L)

Uniform resource locator (U R L) is the common way to find resources on the web. A R U L can be broken down into three main parts:
Scheme: The protocol used, which in this lab will always be http://
Internet address or Base U R L: Used to locate the resource. Examples include

www.ibm.com **and** www.gitlab.com
Route: Location on the web server for example: /images/IDSNlogo.png
You may also hear the term Uniform Resource Identifier (U
RI). U
subset of U
RIs. Another popular term is endpoint, which refers to the U
operation provided by a web server.
R
L are actually a
R
L of an

Request

The process can be broken into the Request and Response process. The request using the get method is partially illustrated below. In the start line we have the GET **method, this is an** HTTP **method. Also the location of the resource** /index.html **and the** HTTP **version. The Request header passes additional information with an** HTTP **request:**

_
Requests

_
HTTP

When an HTTP **request is made, an** HTTP **method is sent, this tells the server what action to perform. A list of several** HTTP **methods is shown below. We will review more examples later.**

Response

The figure below represents the response; the response start line contains the version number HTTP/1.0 **, a status code (200) meaning success, followed by a descriptive phrase (OK). The response header contains usef**
ul information. Finally, we have the
response body containing the requested file, an HTML **document. It should be noted that some requests have headers.**

_
Requests

_
HTTP

Some status code examples are shown in the table below, the prefix indicates the class. These are shown in yellow, with actual status codes shown in white. Check out the following link for more descriptions.

_
Requests

_
HTTP

In [1]:

Install the required libraries

```
!pip install requests
!pip install pillow
```

```
Requirement already satisfied: requests in /opt/conda/lib/python3.12/site-p
ackages (2.32.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /opt/conda/lib/p
ython3.12/site-packages (from requests) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/si
te-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/conda/lib/python
3.12/site-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /opt/conda/lib/python
3.12/site-packages (from requests) (2024.12.14)
Collecting pillow
  Downloading pillow-12.1.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_2
8_x86_64.whl.metadata (8.8 kB)
Downloading pillow-12.1.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_
x86_64.whl (7.0 MB)
```

━━ 7.0/7.0 MB 75.4 MB/s eta 0:00:0
0
Installing collected packages: pillow
Successfully installed pillow-12.1.1

Requests in Python

Requests is a Python Library that allows you to send can import the library as follows:
HTTP/1.1 **requests easily. We**
In [2]:
```
import requests
```
We will also use the following libraries:
In [3]:
```
import os
from PIL import Image
from IPython.display import IFrame
```
You can make a GET **request via the method** get **to www.ibm.com:**
In [4]:
```
url='https://www.ibm.com/'
r=requests.get(url)
```
We have the response object r **, this has information about the request, like the status of the request. You can view the status code using the attribute** status_code **.**
In [5]:
Out[5]:
```
r.status_code
```
200

_
Requests
_
HTTP
You can view the request headers:
In [6]:
```
print(r.request.headers)
```
{'User-Agent': 'python-requests/2.32.3', 'Accept-Encoding': 'gzip, deflate,
br, zstd', 'Accept': '*/*', 'Connection': 'keep-alive', 'Cookie': '_abck=61
58C29B00E1971DC8E04287655FED4B~-1~YAAQqYnMFw3uQ8icAQAAj8rm7A/GecDpJv0uHUYqx
BNf9mW7FKsOK6/WDnIZaT2Q/onfOdcJ68rqY92pSKOK5Yel2bYUgVQWVJFSUqgqktRHJWFzBf1x
UoKfH1qB/2FRYB96adI2ZKPptXgDes+LybMaYuuwroDjRE33RFRtm/BNj7Jq2WvmA4O1Nxsatye
mpnCuugDfGZ6IbqRiMT1/ysESzifqaHPEoPXLUqGoayDyshQGURHeRbtRrbMmfNke+35LvYUn86
qKf/S7ASXHXOND6kAsviBbDIc2qZe83d5EDb309s7DrB7mMxa0aslWGq8CthQNuyOthqgsRlcQZ
DKxAliPYiar/FmDI5ZgJG6kUaTXuN8jGGPSA1HbGG0XXKl8HqiKAIGQeOb1GdlikxwWtZkwvk37
Q/klvf5QhTk1Iy6TsDGtwjh904VVHRx4qMY=~-1~-1~-1~-1~-1; bm_sz=32CD9138AEC99E07
5414246466612C7A~YAAQqYnMFw7uQ8icAQAAj8rm7B+Q8n8wzsj9e+yZvmGTC3kzgr/SOfKoZu
ERotrWGc16Qv8whyvCaF+mSg+OEnQarf3dWrLZvuHHdrLgpRLjFK3XxaLUFZm6TqJpwYJlWyClU
NqFo2akwUCnH19mB+TT8x5r6/06UEKv6Va5odwe/a25+yQl2G0YkQBfcXn+UkuaG4GhpIPqp4EJ
sLP7ZH3+88WH9Ph8FCFUhmQDLOJCJa28B0p9RjOhckTAaRFywCoya29wsMb8GPs6JMDK+/V+C+g
3lU+sTSTEuW3+UlZ2VQht804sjlT/435c24TpodRFb+EArSi0A40yARmrgCsvk1cTrGApN1UL~4
408632~3687472'}
You can view the request body, in the following line, as there is nobody for a get request we get None **:**
In [7]:
```
print("request body:", r.request.body)
```
request body: None
You can view the HTTP **response header using the attribute** headers **. This returns a python dictionary of** HTTP **response headers.**
In [8]:
```
header=r.headers
print(r.headers)
```
{'Content-Security-Policy': 'upgrade-insecure-requests', 'x-frame-options':
'SAMEORIGIN', 'Last-Modified': 'Sat, 14 Mar 2026 15:05:58 GMT', 'ETag': '"3
35c1-64cfd52bada82-gzip"', 'Accept-Ranges': 'bytes', 'Content-Type': 'text/
html;charset=utf-8', 'X-Content-Type-Options': 'nosniff', 'Cache-Control':
'max-age=600', 'Expires': 'Sat, 14 Mar 2026 15:21:14 GMT', 'X-Akamai-Transf
ormed': '0 - 0 -', 'Content-Encoding': 'gzip', 'Date': 'Sat, 14 Mar 2026 1
5:11:14 GMT', 'Content-Length': '41041', 'Connection': 'keep-alive', 'Var

y': 'Accept-Encoding', 'Strict-Transport-Security': 'max-age=31536000'}
You can obtain the date the request was sent using the key Date **.**
In [9]:
Out[9]:
header['date']
'Sat, 14 Mar 2026 15:11:14 GMT'
Content-Type **indicates the type of data:**
In [10…
Out[10]:
header['Content-Type']
'text/html;charset=utf-8'

You can also check the encoding **:**
In [11…
Out[11]:
r.encoding
'utf-8'
As the Content-Type **is** text/html **you can use the attribute** text **to display the** HTML **in the body. You can review the first 100 characters:**
In [12…
Out[12]:
r.text[0:100]
' \n<!DOCTYPE HTML>\n\n<html lang="en">\n<head>\r\n \r\n \r\n \r
\n \r\n \r\n \r\n \r\n \r\n \r\n'
You can load other types of data for non-text requests, like images. Consider the U of the following image:
R
L
In [13…
# Use single quotation marks for defining string
url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM
You can make a get request:
In [14…
r=requests.get(url)
You can look at the response header:
In [15…
print(r.headers)
{'Date': 'Sat, 14 Mar 2026 15:12:41 GMT', 'X-Clv-Request-Id': '8f79cbad-1b0
6-41eb-a7ae-353fbcbf9a39', 'Server': 'Cleversafe', 'X-Clv-S3-Version': '2.
5', 'Accept-Ranges': 'bytes', 'x-amz-request-id': '8f79cbad-1b06-41eb-a7ae-
353fbcbf9a39', 'ETag': '"8bb44578fff8fdcc3d2972be9ece0164"', 'Content-Typ
e': 'image/png', 'Last-Modified': 'Wed, 16 Nov 2022 03:32:41 GMT', 'Content
-Length': '78776'}
You can see the 'Content-Type'
In [16…
Out[16]:
r.headers['Content-Type']
'image/png'
An image is a response object that contains the image as a bytes-like object. As a result, we must save it using a file object. First, you specify the file path and name
In [17…
path=os.path.join(os.getcwd(),'image.png')
You save the file, in order to access the body of the response we use the attribute content **then save it using the** open **f unction and write** method **:**

In [18…

```
with open(path,'wb') as f:
f.write(r.content)
```
You can view the image:

In [19…

Out[19]:

```
Image.open(path)
```
Question: Download a file

Consider the following U

R

L:

URL = <https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-PY0101EN-SkillsNetwork/labs/Module%205/data/Example1.txt>

Write the commands to download the txt file in the given link.

In [22…

```
## Write your code here
url='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM
r=requests.get(url)
with open(path,'wb') as f:
f.write(r.content)
```
Click here for the solution

Get Request with U

R

L Parameters

You can use the GET method to modify the results of your query, for example, retrieving data from an API. We send a GET request to the server. As before, we have the Base U

R

L, in the Route we append /get **, this indicates we would like to preform a** GET **request. This is demonstrated in the following table:**

_

Requests

_

HTTP

In [23…

The Base U

R

L is for http://httpbin.org/ **. It is a simple HTTP Request and Response service. The** URL **in Python is given by:**

```
url_get='http://httpbin.org/get'
```
A query string is a part of a uniform resource locator (U

R

L), it sends other information to the web server. The start of the query is a ? **, followed by a series of parameter and value pairs, as shown in the table below. The first parameter name is** name **and the value is** Joseph **. The second parameter name is** ID **and the Value is** 123 **. Each pair, parameter, and value is separated by an equals sign,** = **. The series of pairs is separated by the ampersand** & **.**

In [24…

In [25…

In [26…

Out[26]:

To create a Query string, add a dictionary. The keys are the parameter names and the values are the value of the Query string.

```
payload={"name":"Joseph","ID":"123"}
```
Then passing the dictionary payload **to the** params **parameter of the** get()

f

unction:

```
r=requests.get(url_get,params=payload)
```

You can print out the URL **and see the name and values.**

r.url

'http://httpbin.org/get?name=Joseph&ID=123'

There is no request body.

In [27…

```
print("request body:", r.request.body)
```

request body: None

You can print out the status code.

In [28…

```
print(r.status_code)
```

200

You can view the response as text:

In [29…

```
print(r.text)
```

{
"args": {
"ID": "123",
"name": "Joseph"
},
"headers": {
"Accept": "*/*",
"Accept-Encoding": "gzip, deflate, br, zstd",
"Host": "httpbin.org",
"User-Agent": "python-requests/2.32.3",
"X-Amzn-Trace-Id": "Root=1-69b57c48-0a3dffa80b2ccdeb7655d04c"
},
"origin": "169.63.179.135",
"url": "http://httpbin.org/get?name=Joseph&ID=123"
}

You can look at the 'Content-Type' **.**

In [30…

Out[30]:

```
r.headers['Content-Type']
```

'application/json'

As the content 'Content-Type' **is in the** JSON **format, you can use the method** json() **, it returns a Python** dict **:**

In [31…

Out[31]:

```
r.json()
```

{'args': {'ID': '123', 'name': 'Joseph'},
'headers': {'Accept': '*/*',
'Accept-Encoding': 'gzip, deflate, br, zstd',
'Host': 'httpbin.org',
'User-Agent': 'python-requests/2.32.3',
'X-Amzn-Trace-Id': 'Root=1-69b57c48-0a3dffa80b2ccdeb7655d04c'},
'origin': '169.63.179.135',
'url': 'http://httpbin.org/get?name=Joseph&ID=123'}

The key args **has the name and values:**

In [32…

```
r.json()['args']
```


Out[32]:

{'ID': '123', 'name': 'Joseph'}

Post Requests

Like a GET **request, a** POST **is used to send data to a server, but the** POST **request sends the data in a request body. In order to send the Post Request in Python, in the**

URL **you can change the route to** POST **:**

In [33…

```
url_post='http://httpbin.org/post'
```

This endpoint will expect data as a file or as a form. A form is convenient way to configure an HTTP request to send data to a server.

To make a POST **request we use the** post() **f unction, the variable** payload **is passed to the parameter** data **:**

In [34…

```
try:
response = requests.post(url_post, data=payload)
if response.status_code == 200:
print("Response JSON:", response.json())
else:
print(f"HTTP Error: {response.status_code} - {response.reason}")
except requests.exceptions.RequestException as e:
print(f"An error occurred: {e}")
```

```
Response JSON: {'args': {}, 'data': '', 'files': {}, 'form': {'ID': '123',
'name': 'Joseph'}, 'headers': {'Accept': '*/*', 'Accept-Encoding': 'gzip, d
eflate, br, zstd', 'Content-Length': '18', 'Content-Type': 'application/x-w
ww-form-urlencoded', 'Host': 'httpbin.org', 'User-Agent': 'python-requests/
2.32.3', 'X-Amzn-Trace-Id': 'Root=1-69b57ced-3b4499552bcd8f8f2dbb0f51'}, 'j
son': None, 'origin': '169.63.179.135', 'url': 'http://httpbin.org/post'}
```

Comparing the U R L from the response object of the GET **and** POST **request you see the** POST **request has no name or value pairs.**

In [35…

```
print("POST request URL:",response.url )
print("GET request URL:",r.url)
```

```
POST request URL: http://httpbin.org/post
GET request URL: http://httpbin.org/get?name=Joseph&ID=123
```

You can compare the POST **and** GET **request body, you see only the** POST **request has a body:**

In [36…

```
print("POST request body:",response.request.body)
print("GET request body:",r.request.body)
```

```
POST request body: name=Joseph&ID=123
GET request body: None
```

You can view the form as well:

_

Requests

HTTP

_

In [37…

Out[37]:

```
response.json()['form']
```

```
{'ID': '123', 'name': 'Joseph'}
```

There is a lot more you can do. Check out Requests for more.

Congratulations, you have completed your hands-on lab on Review Of Accessing APIs.

J

OBS API Implementation Using FLASK

Estimated time needed: 45 to 60 minutes

O

bjectives

You
will be executing this code so that the client application Collecting Jobs API will be accessing this code executing on the server

In [1]: !pip install flask
Requirement already satisfied: flask in /home/jupyterlab/conda/envs/python/li
b/python3.7/site-packages (2.2.3)
Requirement already satisfied: Werkzeug>=2.2.2 in /home/jupyterlab/conda/env
s/python/lib/python3.7/site-packages (from flask) (2.2.3)
Requirement already satisfied: Jinja2>=3.0 in /home/jupyterlab/conda/envs/pyt
hon/lib/python3.7/site-packages (from flask) (3.1.2)
Requirement already satisfied: itsdangerous>=2.0 in /home/jupyterlab/conda/en
vs/python/lib/python3.7/site-packages (from flask) (2.1.2)
Requirement already satisfied: click>=8.0 in /home/jupyterlab/conda/envs/pyth
on/lib/python3.7/site-packages (from flask) (8.1.3)
Requirement already satisfied: importlib-metadata>=3.6.0 in /home/jupyterlab/
conda/envs/python/lib/python3.7/site-packages (from flask) (4.11.4)
Requirement already satisfied: zipp>=0.5 in /home/jupyterlab/conda/envs/pytho
n/lib/python3.7/site-packages (from importlib-metadata>=3.6.0->flask) (3.15.
0)
Requirement already satisfied: typing-extensions>=3.6.4 in /home/jupyterlab/c
onda/envs/python/lib/python3.7/site-packages (from importlib-metadata>=3.6.0-
>flask) (4.5.0)
Requirement already satisfied: MarkupSafe>=2.0 in /home/jupyterlab/conda/env
s/python/lib/python3.7/site-packages (from Jinja2>=3.0->flask) (2.1.1)

Dataset Used in this Assignment

The dataset used in this lab comes from the following source:
https://www.kaggle.com/promptcloud/jobs-on-naukricom under the under a Public Domain license.
Note: We are using a modified subset of that dataset for the lab, so to follow the lab instructions successf
ully please use the dataset provided
with the lab, rather than the dataset from the original source.

_
The original dataset is a csv
of the lab.
. We have converted the csv to json as per the requirement

In [2]: !wget https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IB
--2026-03-14 15:29:42-- https://cf-courses-data.s3.us.cloud-object-storage.a
ppdomain.cloud/IBM-DA0321EN-SkillsNetwork/labs/module%201/Accessing%20Data%20
Using%20APIs/jobs.json
Resolving cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-cour
ses-data.s3.us.cloud-object-storage.appdomain.cloud)... 169.63.118.104, 169.6
3.118.104
Connecting to cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-
courses-data.s3.us.cloud-object-storage.appdomain.cloud)|169.63.118.104|:44
3... connected.
HTTP request sent, awaiting response... 200 OK
Length: 12878382 (12M) [application/json]
Saving to: ‘jobs.json’
jobs.json 100%[===================>] 12.28M 34.4MB/s in 0.4s
2026-03-14 15:29:43 (34.4 MB/s) - ‘jobs.json’ saved [12878382/12878382]

In [ ]:
```
import flask
from flask import request, jsonify
import requests
import re
def get_data(key,value,current):
results = list()
pattern_dict = {
'C' : '(C)',
'C++' : '(C\+\+)',
```

```
'Java' :'(Java)',
'C#' : '(C\#)',
'Python' :'(Python)',
'Scala' : '(Scala)',
'Oracle' : '(Oracle)',
'SQL Server': '(SQL Server)',
'MySQL Server' :'(MySQL Server)',
'PostgreSQL':'(PostgreSQL)',
'MongoDB' : '(MongoDB)',
'JavaScript' : '(JavaScript)',
'Los Angeles' :'(Los Angeles)',
'New York':'(New York)',
'San Francisco':'(San Francisco)',
'Washington DC':'(Washington DC)',
'Seattle':'(Seattle)',
'Austin':'(Austin)',
'Detroit':'(Detroit)',
```

```
}
for rec in current:
print(rec[key])
print(type(rec[key]))
print(rec[key].find(value))
#if rec[key].find(value) != -1:
import re
#reex_str = """(C)|(C\+\+)|(JavaScript)|(Java)|(C\#)|(Python)|(Scala
if re.search(pattern_dict[value],rec[key]) != None:
results.append(rec)
return results
app = flask.Flask(__name__)
import json
data = None
with open('jobs.json',encoding='utf-8') as f:
# returns JSON object as
# a dictionary
data = json.load(f)
@app.route('/', methods=['GET'])
def home():
return '''<h1>Welcome to flask JOB search API</p>'''
@app.route('/data/all', methods=['GET'])
def api_all():
return jsonify(data)
@app.route('/data', methods=['GET'])
def api_id():
# Check if keys such as Job Title,KeySkills, Role Category and others a
# Assign the keys to the corresponding variables..
# If no key is provided, display an error in the browser.
res = None
for req in request.args:
if req == 'Job Title':
key = 'Job Title'
elif req == 'Job Experience Required' :
key='Job Experience Required'
elif req == 'Key Skills' :
key='Key Skills'
elif req == 'Role Category' :
key='Role Category'
elif req == 'Location' :
key='Location'
elif req == 'Functional Area' :
key='Functional Area'
```

```
elif req == 'Industry' :
key='Industry'
elif req == 'Role' :
key='Role'
elif req=="id":
key="id"
else:
pass
value = request.args[key]
if (res==None):
res = get_data(key,value,data)
else:
res = get_data(key,value,res)
# Use the jsonify function from Flask to convert our list of
# Python dictionaries to the JSON format.
return jsonify(res)
app.run()
* Serving Flask app '__main__'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployme
nt. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Autho

Collecting Job Data Using APIs

Estimated time needed: 30 minutes

O

bjectives

After completing this lab, you will be able to:

Collect job data using Jobs API

Store the collected data into an excel spreadsheet.

Note: Before starting with the assignment make sure to read all the instructions and then move ahead with the coding part.

Instructions

To run the actual lab, firstly you need to click on the Jobs_API notebook link. The file contains flask code which is required to run the Jobs API data.

Now, to run the code in the file that opens up follow the below steps.

Step1: Download the file.

Step2: Upload the file into your current Jupyter environment using the upload button in your Jupyter interface. Ensure that the file is in the same folder as your working .ipynb file.

Step 2: If working in a local Jupyter environment, use the "Upload" button in your Jupyter interface to upload the Jobs_API notebook into the same folder as your current .ipynb file.

Step3: Open the Jobs_API notebook, and run all the cells to start the Flask application. Once the server is running, you can access the API from the U R

L provided in the
notebook.
If you
want to learn more about flask, which is optional, you can click on this link here.
Once you run the flask code, you can start with your assignment.

Dataset Used in this Assignment

The dataset used in this lab comes from the following source:
https://www.kaggle.com/promptcloud/jobs-on-naukricom under the under a Public Domain license.
Note: We are using a modified subset of that dataset for the lab, so to follow the lab instructions successf ully please use the dataset provided with the lab, rather than the dataset from the original source.
The original dataset is a csv of the lab.
. We have converted the csv to json as per the requirement

Warm-Up Exercise

Before you attempt the actual lab, here is a f ully solved warmup exercise that will help you to learn how to access an API.
Using an API, let us find out who currently are on the International Space Station (ISS).
The API at http://api.open-notify.org/astros.json gives us the information of astronauts currently on ISS in json format.
You can read more about this API at http://open-notify.org/Open-Notify-API/People-In-Space/

In [1]:
```
import requests # you need this module to make an API call
import pandas as pd
```
In [2]:
```
api_url = "http://api.open-notify.org/astros.json" # this url gives use the
```
In [3]:
```
response = requests.get(api_url) # Call the API using the get method and sto
# output of the API call in a variable calle
```
In [4]:
```
if response.ok: # if all is well() no errors, no network timeout
data = response.json() # store the result in json format in a variable
# the variable data is of type dictionary.
```
In [5]:
```
print(data) # print the data just to check the output or for debugging
```
```
{'people': [{'craft': 'ISS', 'name': 'Oleg Kononenko'}, {'craft': 'ISS', 'nam
e': 'Nikolai Chub'}, {'craft': 'ISS', 'name': 'Tracy Caldwell Dyson'}, {'craf
t': 'ISS', 'name': 'Matthew Dominick'}, {'craft': 'ISS', 'name': 'Michael Bar
ratt'}, {'craft': 'ISS', 'name': 'Jeanette Epps'}, {'craft': 'ISS', 'name':
'Alexander Grebenkin'}, {'craft': 'ISS', 'name': 'Butch Wilmore'}, {'craft':
'ISS', 'name': 'Sunita Williams'}, {'craft': 'Tiangong', 'name': 'Li Guangs
u'}, {'craft': 'Tiangong', 'name': 'Li Cong'}, {'craft': 'Tiangong', 'name':
'Ye Guangfu'}], 'number': 12, 'message': 'success'}
```

Print the number of astronauts currently on ISS.

In [6]:
```
print(data.get('number'))
```



```
12
```

Print the names of the astronauts currently on ISS.

In [7]:
```
astronauts = data.get('people')
print("There are {} astronauts on ISS".format(len(astronauts)))
print("And their names are :")
for astronaut in astronauts:
print(astronaut.get('name'))
```
```
There are 12 astronauts on ISS
```

And their names are :
Oleg Kononenko
Nikolai Chub
Tracy Caldwell Dyson
Matthew Dominick
Michael Barratt
Jeanette Epps
Alexander Grebenkin
Butch Wilmore
Sunita Williams
Li Guangsu
Li Cong
Ye Guangfu
Hope the warmup was helpf
ul. Good luck with your next lab!

Lab: Collect Jobs Data using Jobs API

O

bjective: Determine the number of jobs currently open for various technologies and for various locations

Collect the number of job postings for the following locations using the API:
Los Angeles
New York
San Francisco
Washington DC
Seattle
Austin
Detroit

In [8]: #Import required libraries
import pandas as pd
import json

https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBM-DA0321EN-SkillsNetwork/labs/module%201/Accessing%20Data%20Using%20APIs/jobs.json

Write a f

unction to get the number of jobs for the Python technology.

Note:
While completing this exercise, first retrieve the f
ull dataset from the API endpoint.
After retrieving the data, apply the required filtering and calculations locally in Python based on the given parameters.
The keys in the json are
Job Title
Job Experience Required
Key Skills
Role Category
Location
Functional Area
Industry
Role
You can also view the json file contents from the following json U
R
L.

In [38]: api_url="https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
def get_number_of_jobs_T(technology):
#your code goes here
number_of_jobs = 0

```
#fetch data from API
response= requests.get(api_url)
if response.ok:
data = response.json()
# loop through each job entry in JSON file
for job in data:
# look for technology in the 'Key Skills'
# lower will count search term if upper, lower, or mixed case
if technology.lower() in job.get('Key Skills',
'').lower():
number_of_jobs+=1
return technology,number_of_jobs
```

Calling the f
unction for Python and checking if it works.

```
In [37]: get_number_of_jobs_T("Python")
Out[37]:
('Python', 1173)
```

Write a f
unction to find number of jobs in US for a location of your choice


```
In [19]: def get_number_of_jobs_L(location):
number_of_jobs = 0
#fetch data from API
response= requests.get(api_url)
if response.ok:
data = response.json()
# loop through each job entry in JSON file
for loc in data:
# look for location in the 'Location'
# lower will count search term if upper, lower, or mixed case
if location.lower() in loc.get('Location',
'').lower():
number_of_jobs+=1
return location,number_of_jobs
```

Call the f
unction for Los Angeles and check if it is working.

```
In [22]: #your code goes here
get_number_of_jobs_L('Washington DC')
Out[22]:
('Washington DC', 5316)
```

Store the results in an excel file

Call the API for all the given technologies below and write the results in an excel spreadsheet.
Technologies:
C
C#
C++
Java
JavaScript
Python
Scala
Oracle
SQL Server
mysql
PostgreSQL
MongoDB
If you do not know how create excel file using python, double click here for hints.
Create a python list of all technologies for which you need to find the number of jobs postings.

In [35]: #your code goes here
technologies = ["C", "C#", "C++", "Java", "JavaScript", "Python", "Scala",
"

Import libraries required to create excel spreadsheet

In [31]: # your code goes here
import pandas as pd
import requests
!pip install openpyxl
from openpyxl import Workbook
Requirement already satisfied: openpyxl in /home/jupyterlab/conda/envs/pytho
n/lib/python3.7/site-packages (3.1.3)
Requirement already satisfied: et-xmlfile in /home/jupyterlab/conda/envs/pyth
on/lib/python3.7/site-packages (from openpyxl) (1.1.0)

Create a workbook and select the active worksheet

In [32]: # your code goes here
# create a new workbook instance
wb = Workbook ()
# select active worksheet
ws = wb.active

Find the number of jobs postings for each of the technology in the above list. Write the technology name and the number of jobs postings into the excel spreadsheet.

In [40]: #your code goes here
def get_all_job_counts(technology):
number_of_jobs = 0
#fetch data from API
response = requests.get(api_url)
if response.ok:
data = response.json()
# loop through each job entry in JSON file
for job in data:
# look for technology in the 'Key Skills'
# lower will count search term if upper, lower, or mixed case
if technology.lower() in job.get('Key Skills',
').lower():
number_of_jobs+=1
return technology, number_of_jobs
# Collect results by looping through data and appending pre-made workbook
for tech in technologies:
tech_name, count = get_all_job_counts(tech)
ws.append([tech_name, count]) #adds a new row to excel sheet

Save into an excel spreadsheet named job-postings.xlsx.

In [41]: #your code goes here
#save updated workbook with name

wb.save('job-postings.xlsx')

In the similar way, you can try for below given technologies and results can be stored in an excel sheet.

Collect the number of job postings for the following languages using the API:

C
C#
C++
Java
JavaScript
Python
Scala

Oracle
SQL Server
MySQL Server
PostgreSQL
MongoDB

```
In [46]: # your code goes here
#define parameters
technologies_2 = ["C", "C#", "C++", "Java", "JavaScript", "Python", "Scala",
# create a new workbook instance
wb = Workbook ()
# select active worksheet
ws = wb.active
def get_all_job_counts(technology):
number_of_jobs = 0
#fetch data from API
response = requests.get(api_url)
if response.ok:
data = response.json()
# loop through each job entry in JSON file
for job in data:
# look for technology in the 'Key Skills'
# lower will count search term if upper, lower, or mixed case
if technology.lower() in job.get('Key Skills',
'').lower():
number_of_jobs+=1
return technology, number_of_jobs
# Collect results by looping through data and appending pre-made workbook
for tech in technologies_2:
tech_name, count = get_all_job_counts(tech)
ws.append([tech_name, count]) #adds a new row to excel sheet
```



```
wb.save('job-postings_2.xlxs')
```

GitHub Overview

First, let's introduce you to GitHub. GitHub is a collection of folders and files. It is a Git repository hosting service, but it adds many of its own features. Git is a command-line tool. It hosts and maintains a server via command line. GitHub provides this Git server and a Web-based graphical interface for you. It also provides access control and collaboration features, such as wikis and basic task management tools for every project. In addition, GitHub provides cloud storage for source code, supports all popular programming languages, and streamlines the iteration process. GitHub includes a free plan for individual developers and hosting Open Source projects.

Hands-on Lab: Getting Started with GitHub

Estimated time: 20 min

In this lab, you will get started with GitHub by creating a GitHub account and project and adding a file to it using its Web interface.

Objectives

After completing this lab, you will be able to:

1. Describe GitHub
2. Create a GitHub account
3. Add a project and repo
4. Edit and create a file
5. Upload a file and Commit

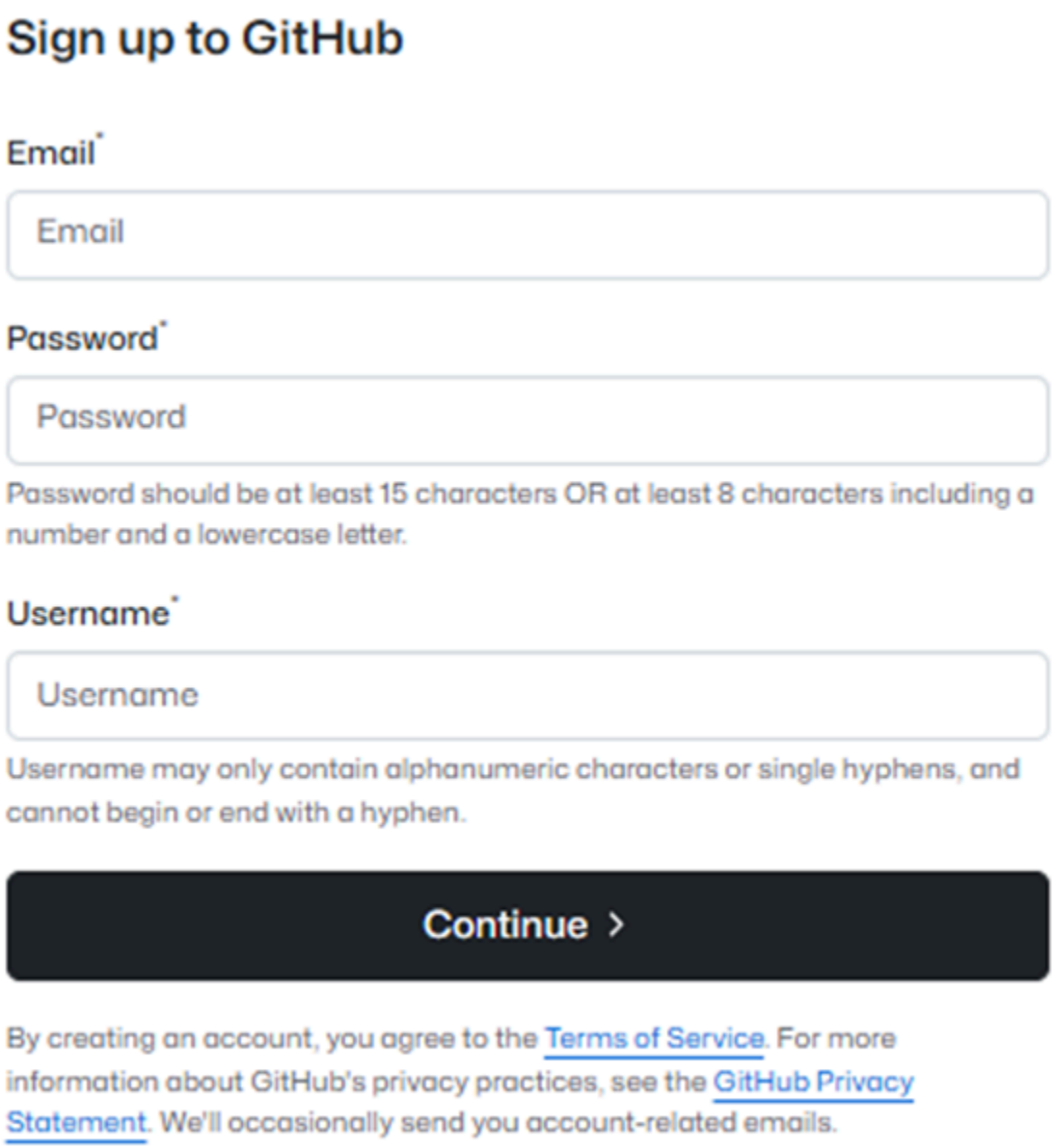
GitHub Overview

First, let's introduce you to GitHub. GitHub is a collection of folders and files. It is a Git repository hosting service, but it adds many of its own features. Git is a command-line tool. It hosts and maintains a server via command line. GitHub provides this Git server and a Web-based graphical interface for you. It also provides access control and collaboration features, such as wikis and basic task management tools for every project. In addition, GitHub provides cloud storage for source code, supports all popular programming languages, and streamlines the iteration process. GitHub includes a free plan for individual developers and hosting Open Source projects.

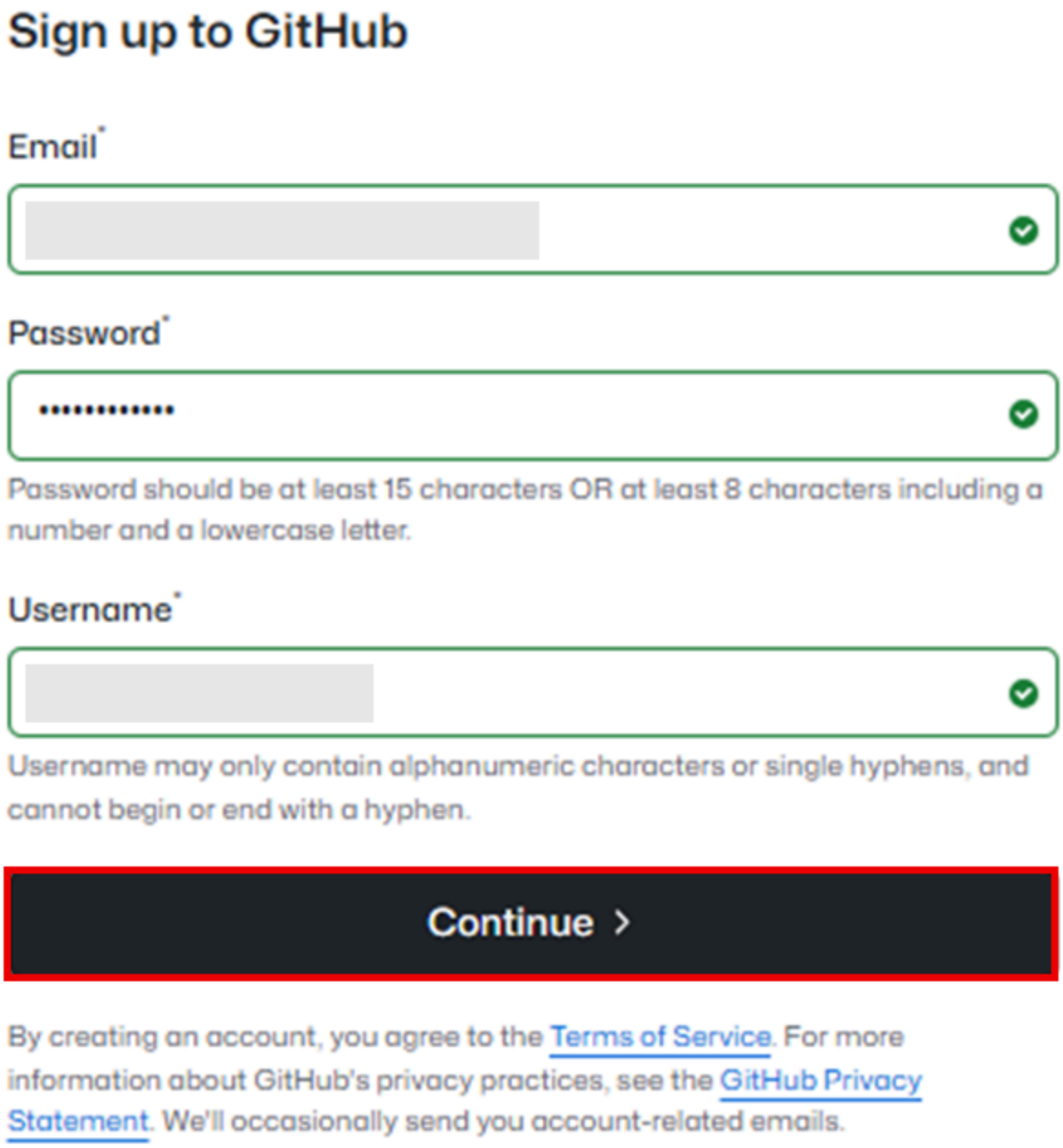
Exercise 1: Creating a GitHub Account

Please use the following steps to create an account on GitHub:

Step 1: Create an account: https://github.com/join

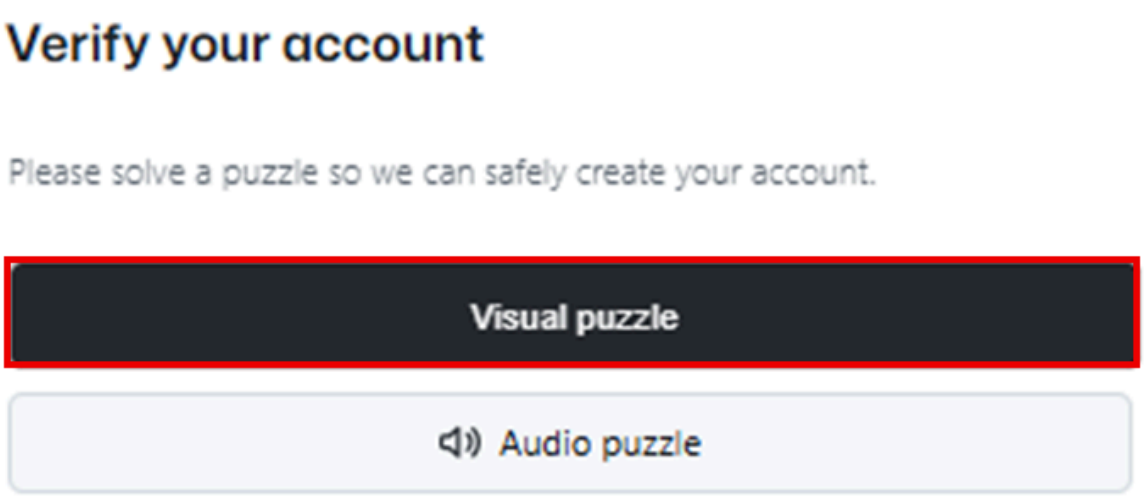


NOTE: If you already have a GitHub account, you can skip this step and simply log in to your account.
Step 2: Provide the necessary details to create an account as shown below:

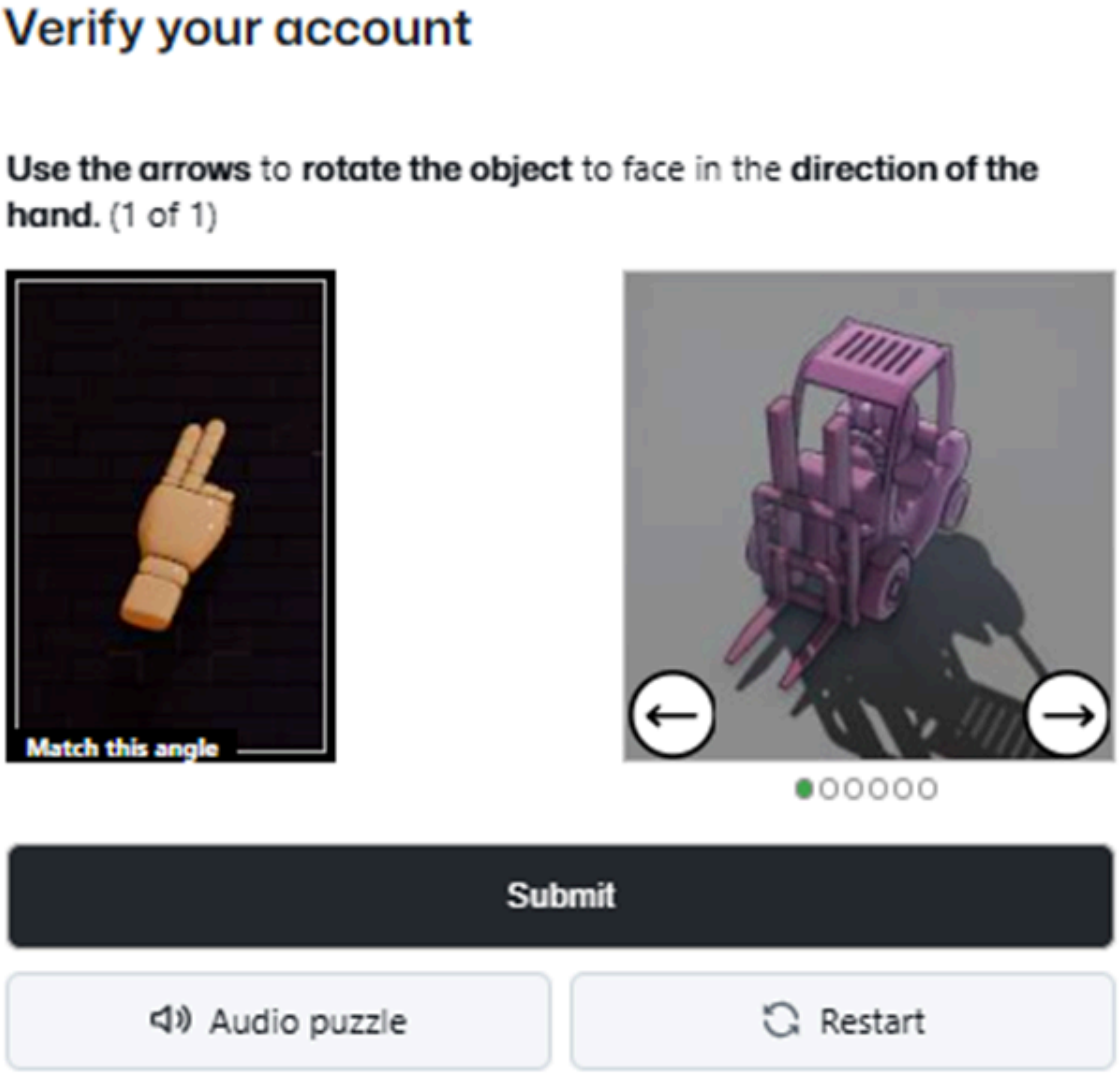


Click Continue.

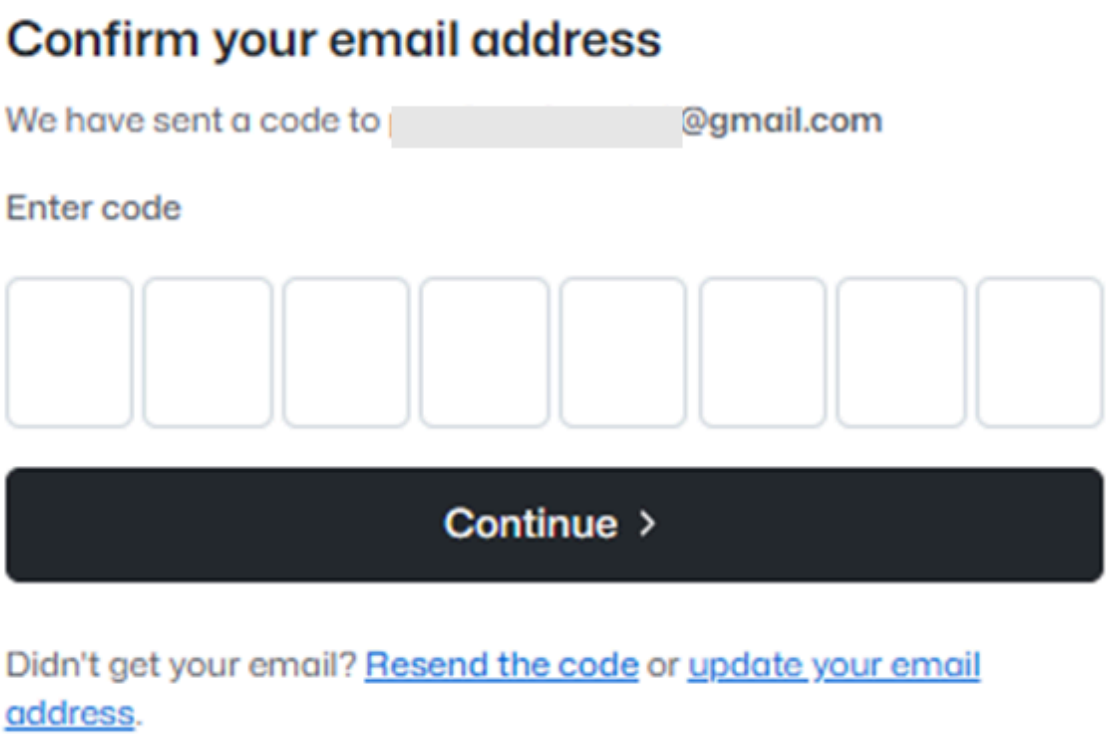
Step 3: Click Visual puzzle to verify the account.



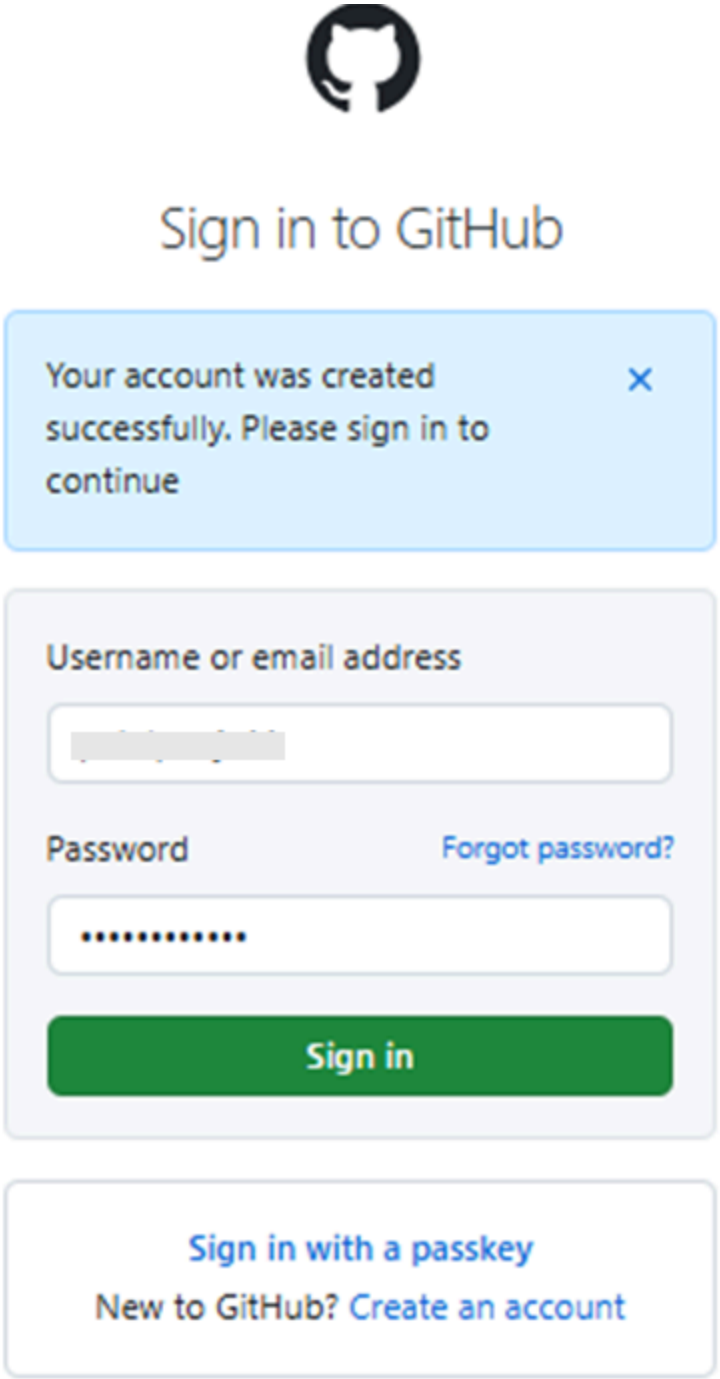
Step 4: Solve the puzzle and Submit.



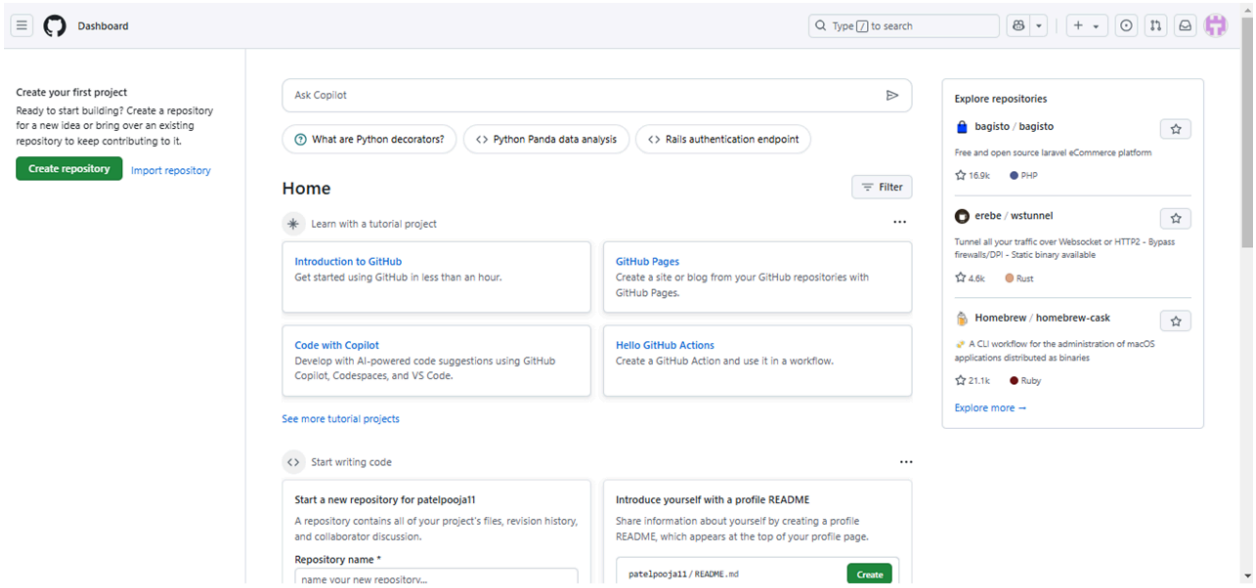
Step 5: Open your email, find the GitHub verification email, copy the verification code, and return to the GitHub Signup page to enter it.
Step 6: Confirm youe eamil address using the verification code and Continue.



NOTE: If you do not receive the verification email, click Resend the code.
Step 7: Sign in to GitHub, enter Id and Password and Sign in

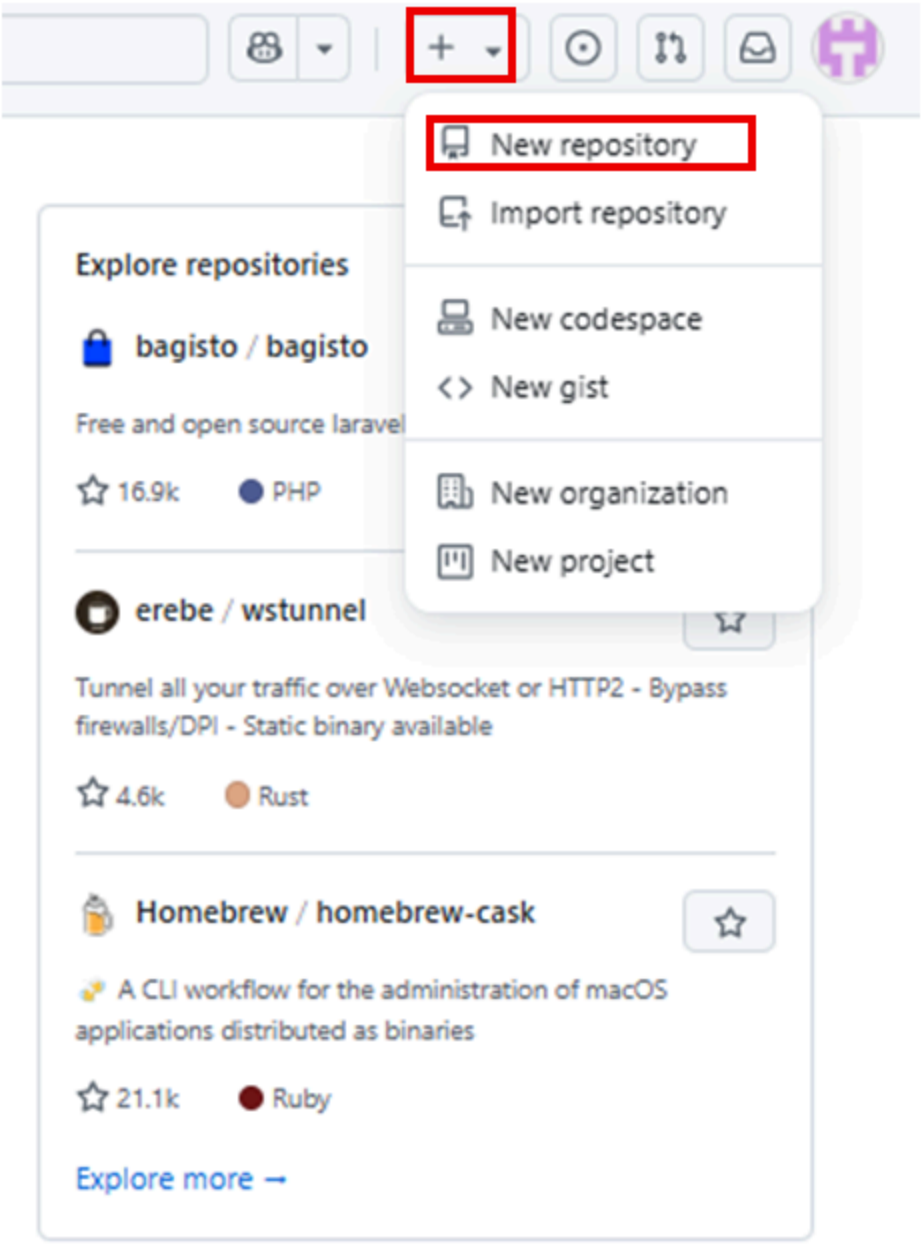


You will see the home page.

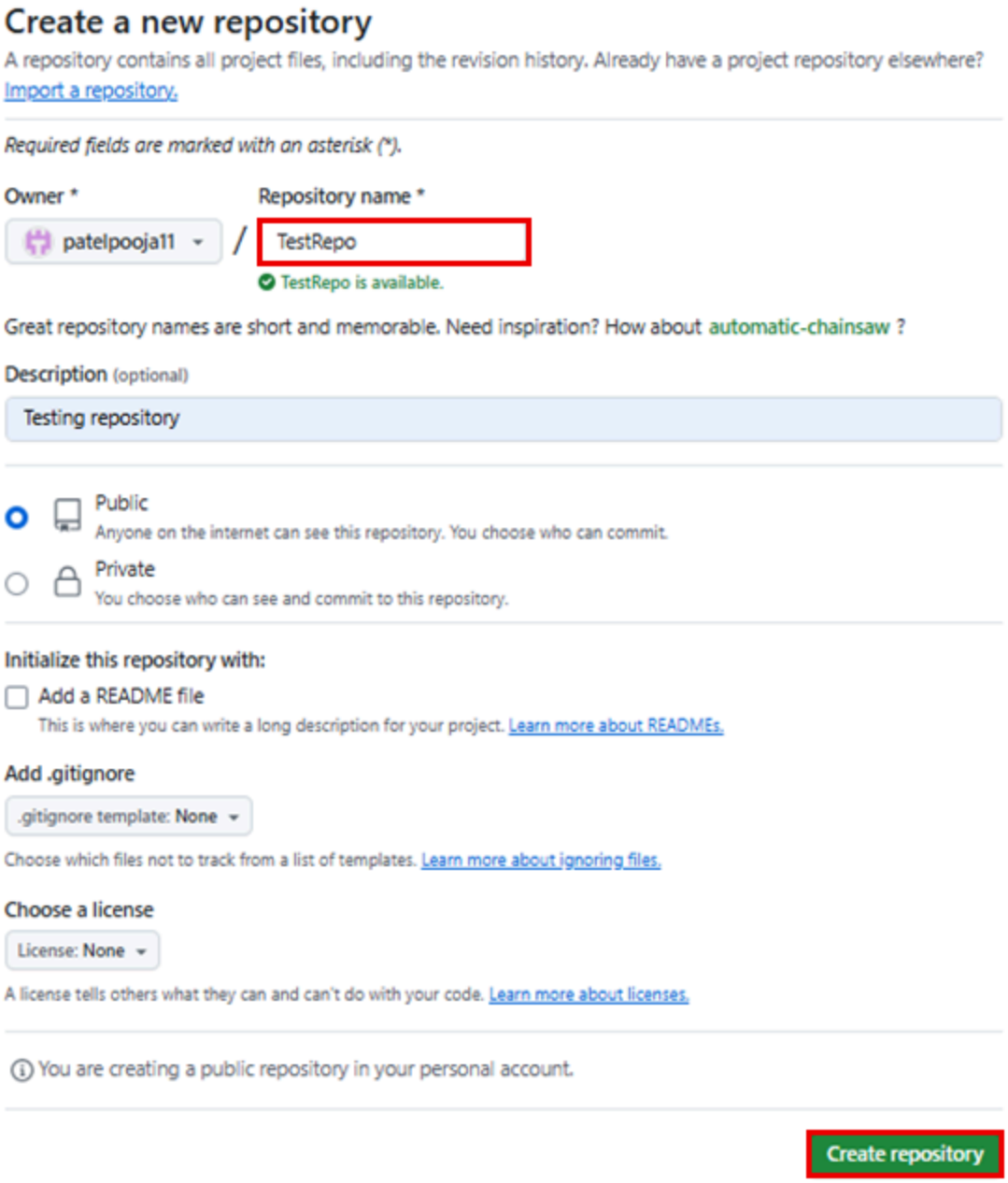


Exercise 2: Adding a project and repo

Step 1: Click the + symbol and click New repository.



Step 2: Provide a name for the repository and initialize it with the empty README.md file.

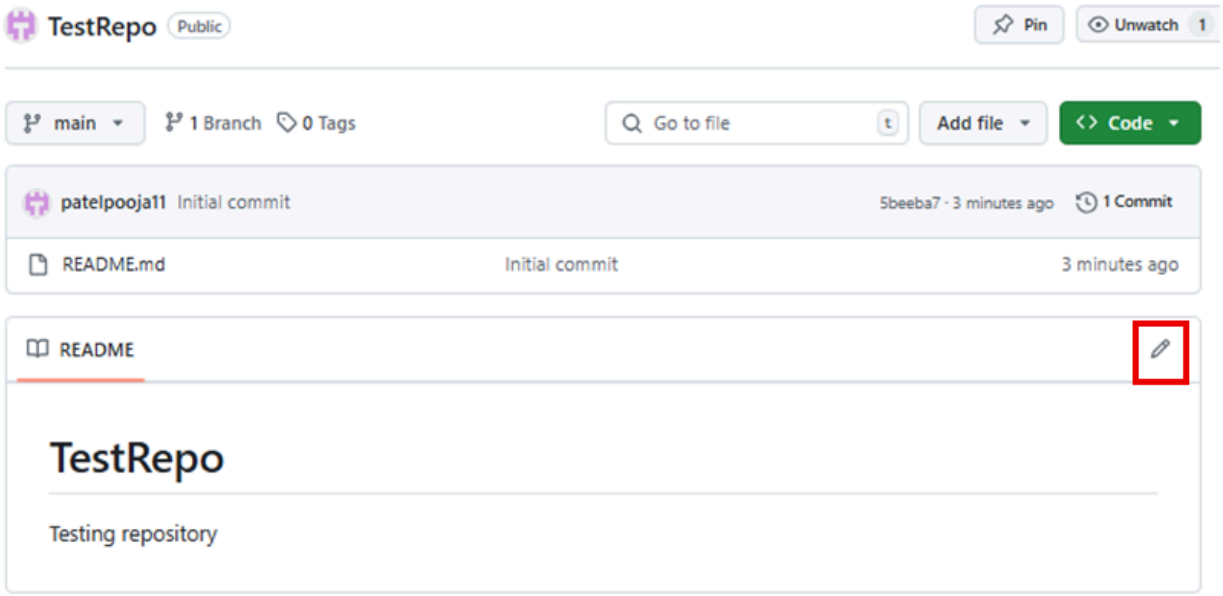


Click Create repository.
Now, you will be redirected to the repository you have created.
Let us start editing the repository.

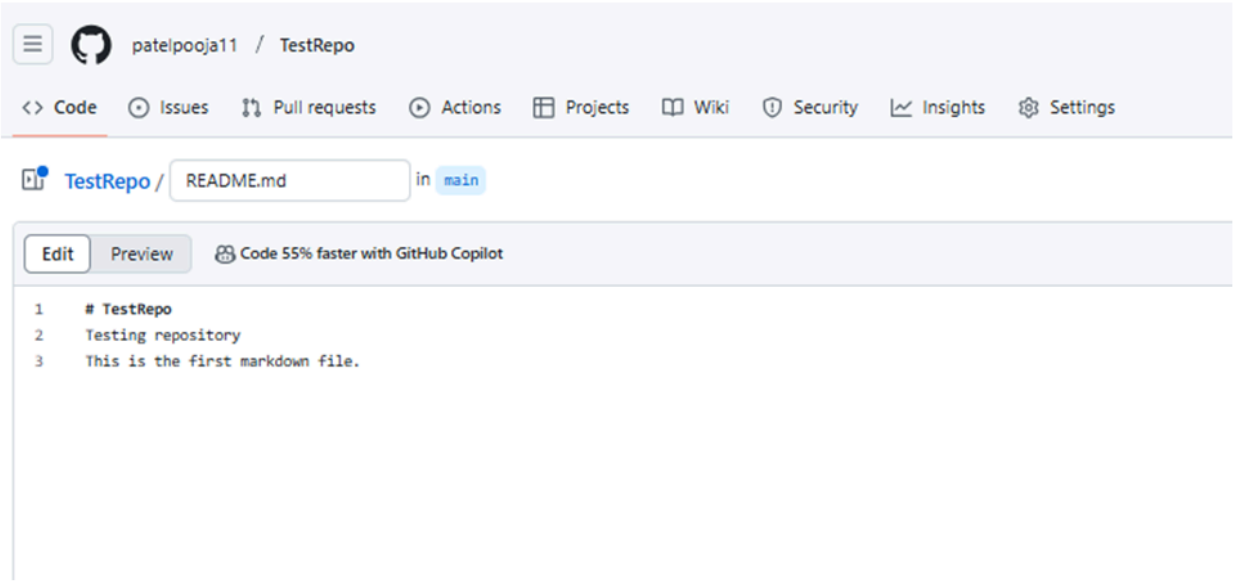
Exercise 3: Create and edit a file

Exercise 3a: Edit a file

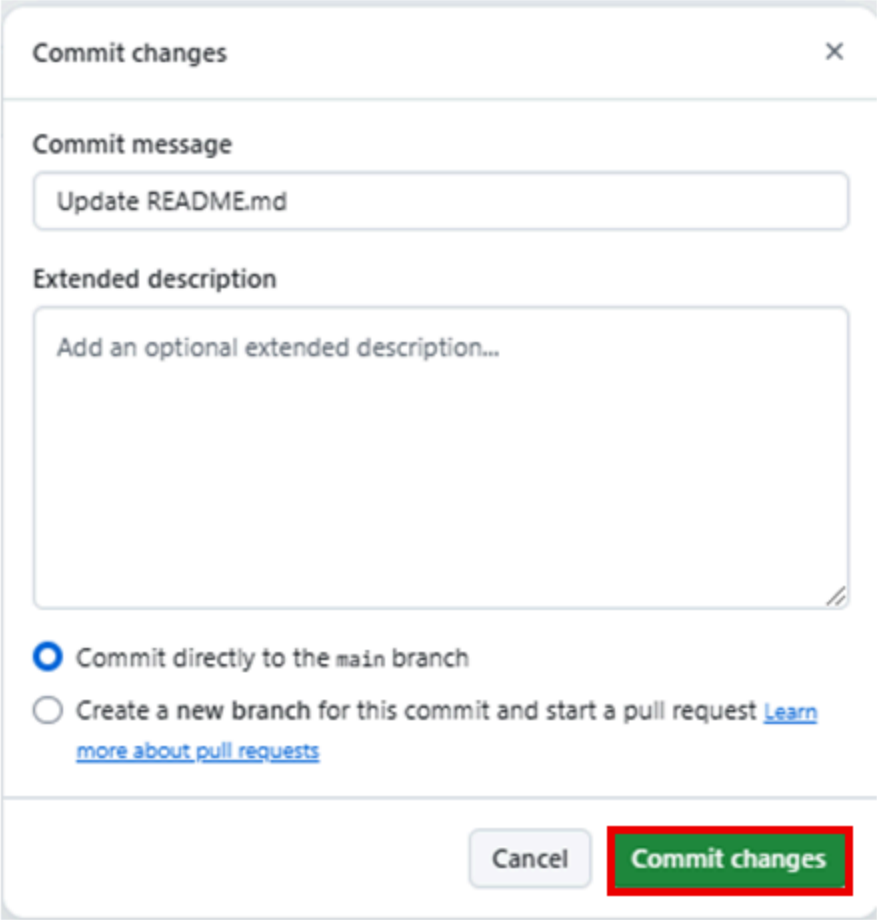
Step 1: Once the repository is created, the root folder of your repository is listed by default, and has just one file, ReadMe.md. Click the pencil icon to edit the file.



Step 2: Add some text to the file.



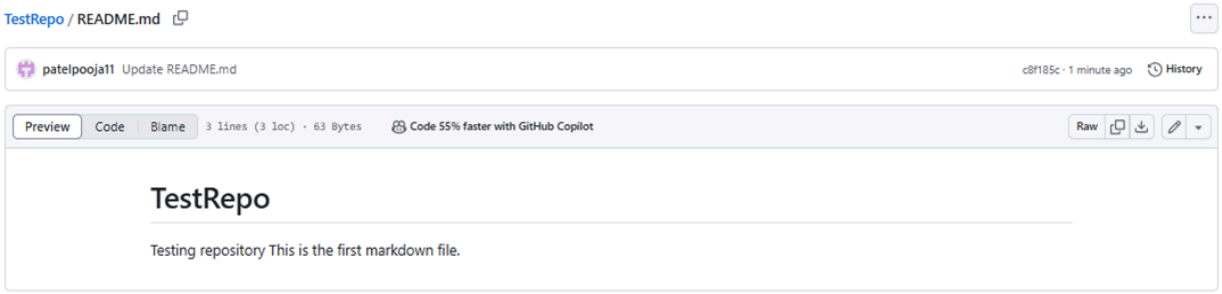
Step 3: After adding the text and click Commit Changes.



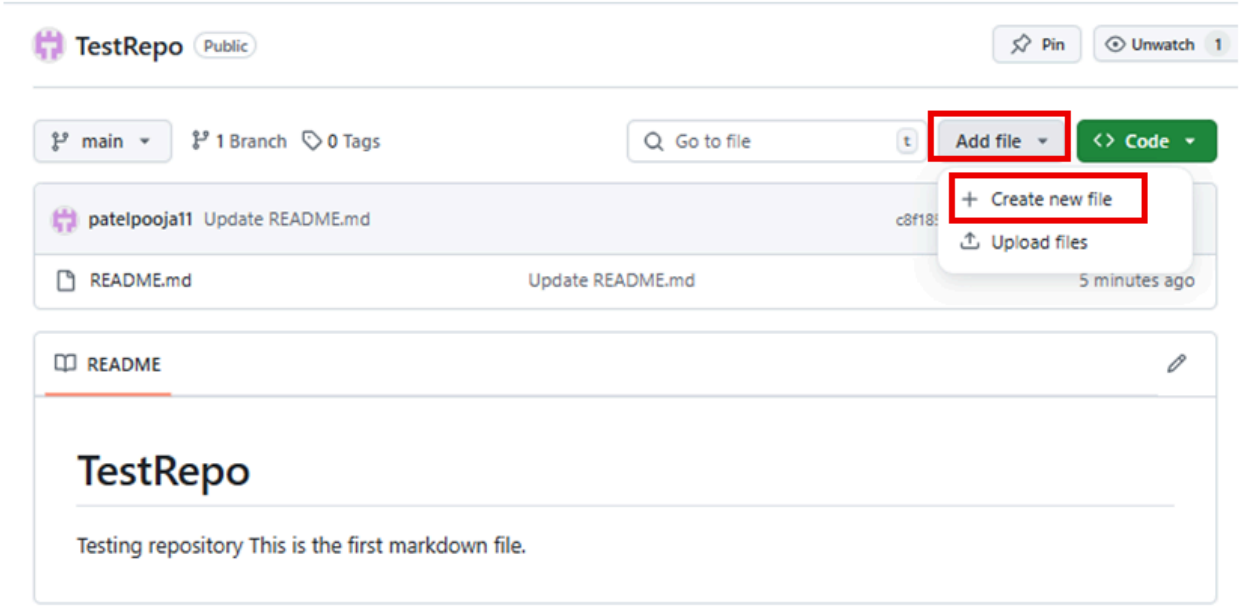
Now, check that your file is edited with the new text.

Exercise 3b: Create a new file

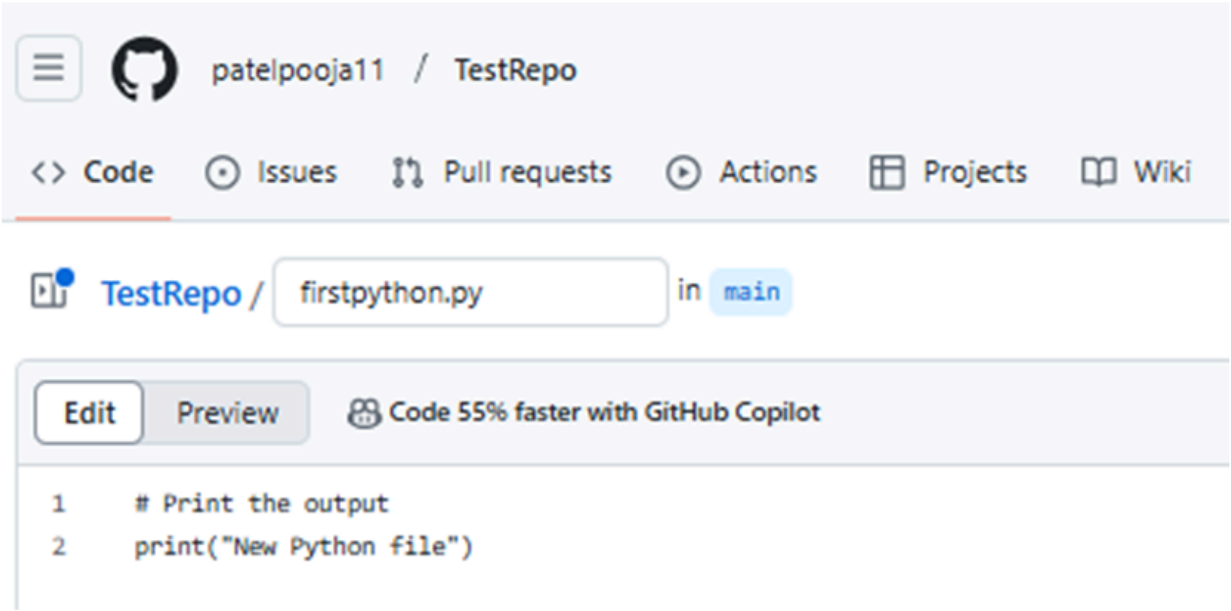
Step 1: Click the repository name to return to the master branch, like in this testrepo.



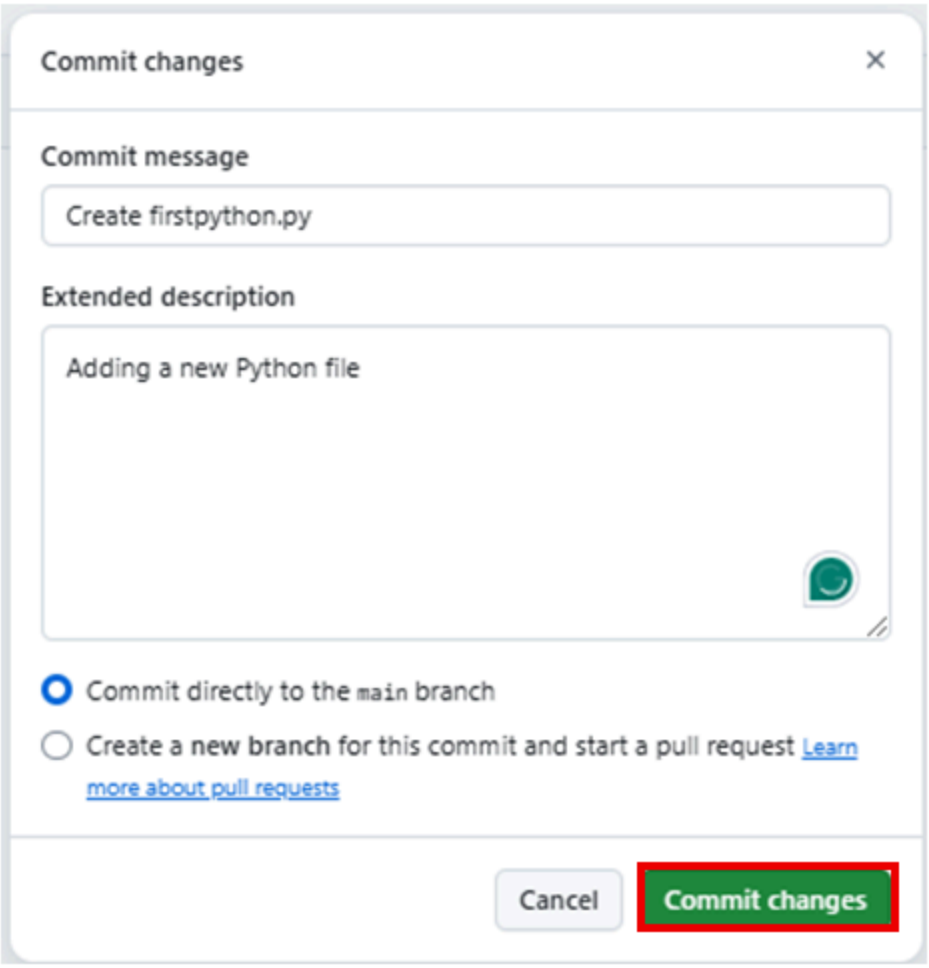
Step 2: Click Add file and select Create New file to create a file in the repository.



Step 3: Provide the file name and the extension of the file. For example, firstpython.py and add the lines.



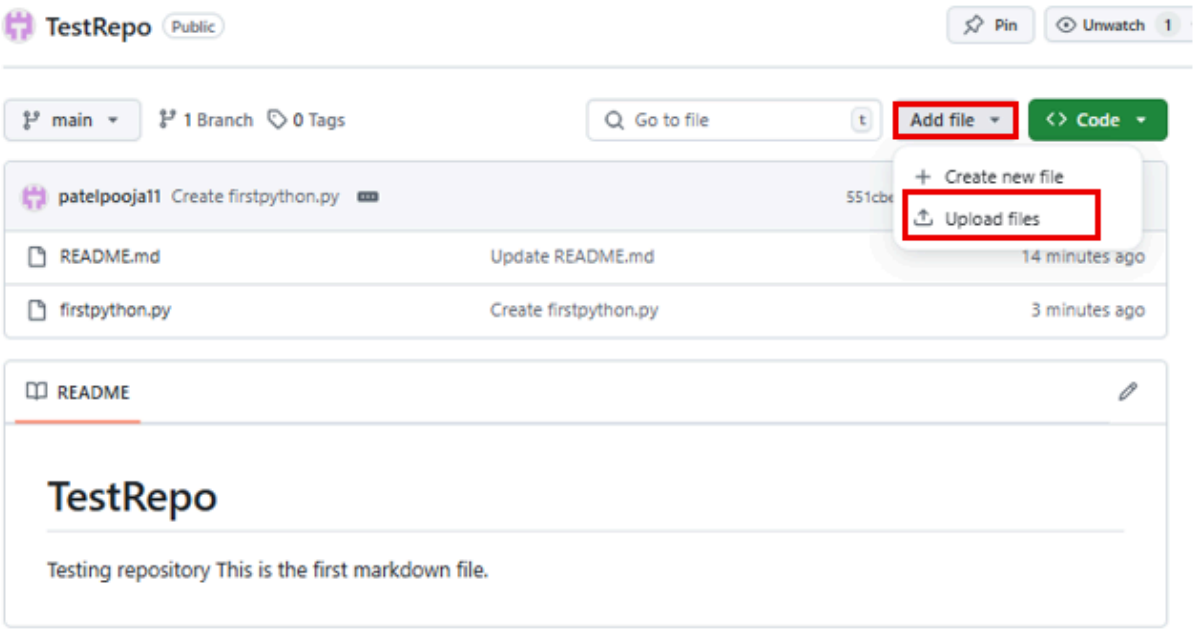
Step 4: Commit changes after adding the text. Add description of the file (optional) and click Commit changes.



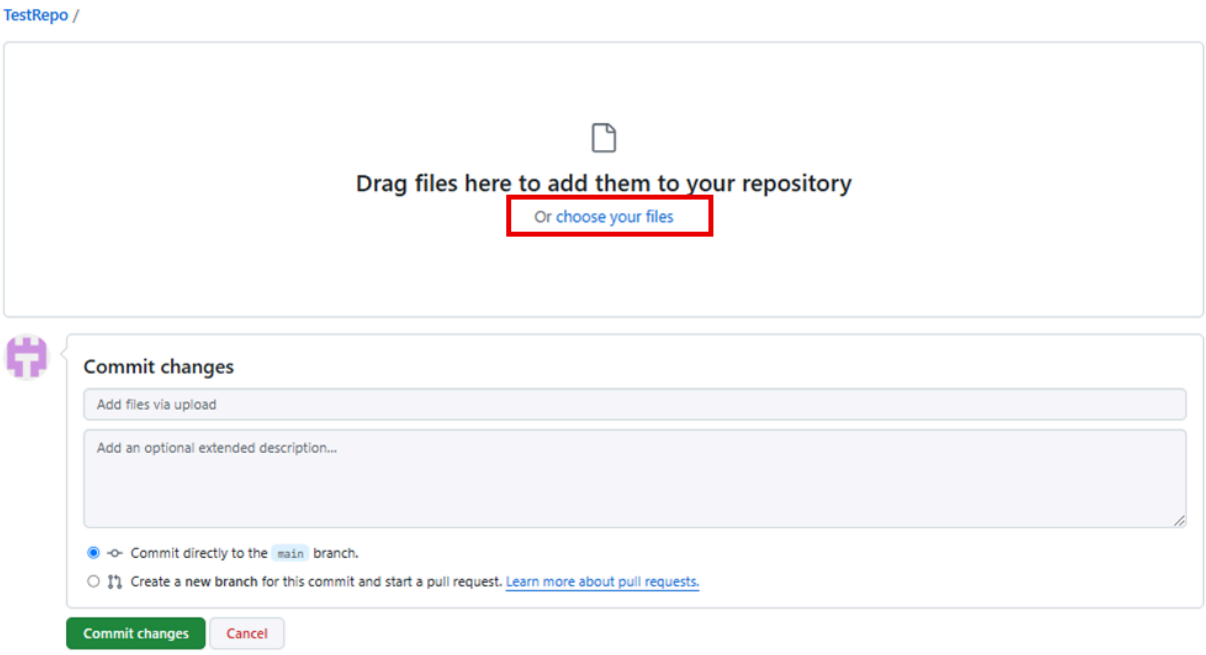
Step 5: Your file is now added to your repository, and the repository listing shows when the file was added and changed.

Exercise 4: Upload a file & Commit

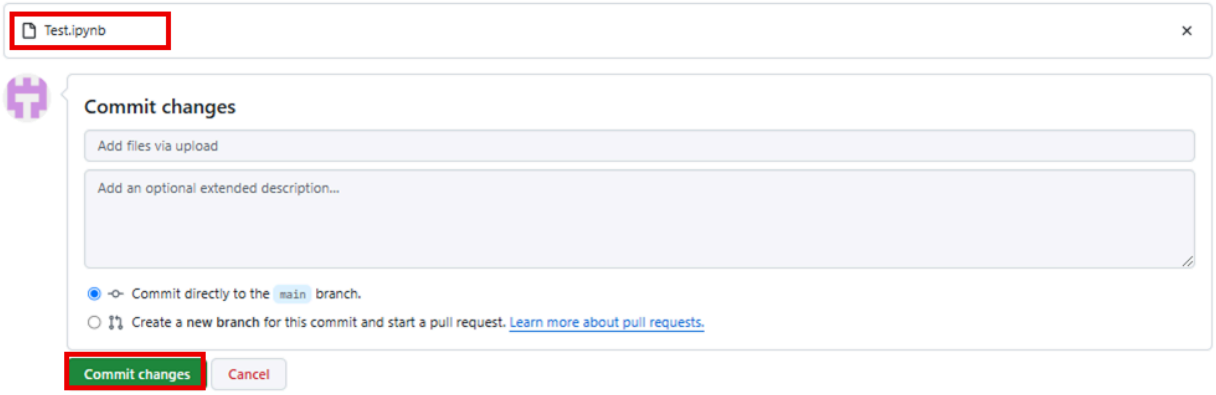
Step 1: Click Add file and select Upload files to upload a file (any .txt, .ipynb, .png file) in the repository from the local computer.



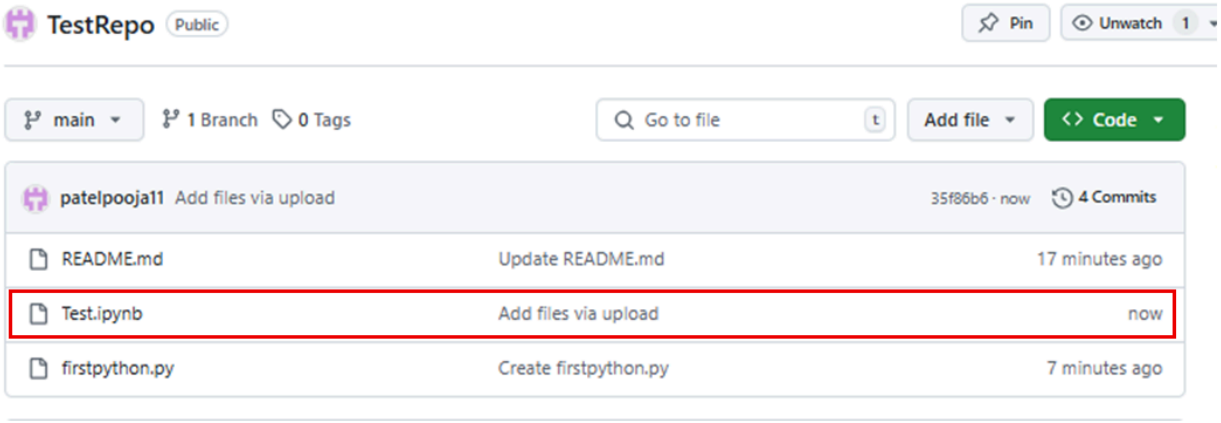
Step 2: Click choose your files and select any files from your computer.



Step 3: Once the file finishes uploading, click Commit changes.



Step 4: Now, your file is uploaded in the repository.



Summary

In this document, you have learned how to create a new repository, add a new file, edit a file, upload a file in a repository, and commit the changes.

Have you uploaded the Jobs_API file to IBM Watson Studio or SN Labs, and run all the cells?

Question 7
Have you called the GitHub Jobs API to collect job postings for Python technology?

Question 10
Have you collected job data for all required technologies and saved it in 'github-job-postings.xlsx'?

Lab: About the Dataset

Estimated time: 20 minutes

Learning Objectives

After completing this lab, you will be able to:

- Summarize the key findings and trends from the Stack Overflow Developer Survey dataset.

Introduction

Stack Overflow, the go-to platform for developers, conducted a global survey to capture insights into the developer community. This survey includes various questions related to professional experience, coding activities, tools, technologies, and preferences. The dataset offers important information about the current state of software development worldwide.
In this lab, you will work with a subset of the original dataset to explore, analyze, and visualize various trends. The survey data consists of responses from developers with different professional backgrounds, educational levels and geographic locations.

Dataset Source

The dataset is available as part of the Stack Overflow Developer Survey under an Open Database License (ODbL). You can access the full data set at this link.

Subset Information

For this lab, you will use a subset of the full data set, which contains several thousand responses. Please note that the conclusions drawn from this subset may not fully reflect the global developer community.

Column Description for Survey Data

Column name	Question text
ResponseId	Randomized respondent ID number.
MainBranch	Which of the following options best describes you today?
Age	What is your age?
Employment	What is your current employment status?

RemoteWork	How often do you work remotely?
Check	Check Various verification or check questions related to survey consistency.
CodingActivities	What coding activities do you engage in (hobby, professional, and open-source contributions)?
EdLevel	What is the highest level of formal education you have completed?
LearnCode	How did you learn to code?
LearnCodeOnline	Have you used online resources to learn coding?
TechDoc	How do you use technical documentation?
YearsCode	How many years have you been coding?
YearsCodePro	How many years have you coded professionally?
DevType	What is your role or type of development work you do?
OrgSize	What is the size of the organization you work for?
PurchaseInfluence	How much influence do you have on purchasing technology at your company?
BuyNewTool	How does your company decide whether to buy new tools or technology?
BuildvsBuy	Does your company prefer to build or buy software?
TechEndorse	Do you endorse any specific technologies at your company?
Country	In which country do you reside?
Currency	Which currency do you use day-to-day?
CompTotal	What is your current total compensation (salary, bonuses, and so on)?
LanguageHaveWorkedWith	Which programming languages have you worked with in the past year?
LanguageWantToWorkWith	Which programming languages do you want to work with in the future?
LanguageAdmired	Which programming languages do you admire most?

DatabaseHaveWorkedWith	Which database technologies have you worked with in the past year?
DatabaseWantToWorkWith	Which database technologies do you want to work with in the future?
DatabaseAdmired	Which database technologies do you admire most?
PlatformHaveWorkedWith	Which platforms have you worked with in the past year?
PlatformWantToWorkWith	Which platforms do you want to work with in the future?
PlatformAdmired	Which platforms do you admire most?
WebframeHaveWorkedWith	Which web frameworks have you worked with in the past year?
WebframeWantToWorkWith	Which web frameworks do you want to work with in the future?
WebframeAdmired	Which web frameworks do you admire most?
EmbeddedHaveWorkedWith	Which embedded systems have you worked with in the past year?
EmbeddedWantToWorkWith	Which embedded systems do you want to work with in the future?
EmbeddedAdmired	Which embedded systems do you admire most?
MiscTechHaveWorkedWith	Which miscellaneous technologies have you worked with in the past year?
MiscTechWantToWorkWith	Which miscellaneous technologies do you want to work with in the future?
MiscTechAdmired	Which miscellaneous technologies do you admire most?
OpSysPersonal	What operating systems do you use for personal tasks?
OpSysProfessional	What operating systems do you use for professional tasks?
SOVisitFreq	How frequently do you visit Stack Overflow?
SOAccount	Do you have a Stack Overflow account?
SOPartFreq	How often do you participate in Q&A on Stack Overflow?

AISelect	How do you feel about artificial intelligence tools for development?
AIBen	What benefits have you experienced from using AI tools?
AIChallenges	What challenges have you faced while using AI tools?
JobSat	How satisfied are you with your current job?

Summary

After completing this lab, you will be able to:

- Recognize the dataset is a valuable resource for analyzing trends in software development.
- Gain insights into the most popular programming languages, platforms, and frameworks, as well as the challenges and opportunities facing developers today by exploring the survey data.